Source: docs\DELIVERY_REVIEW.md | Export: 11 November 2026

> **Current shipping status (11 November 2026):** `DISTANCE_DELIVERY_ENABLED=false` by default. New checkout uses the existing saved flat shipping configuration, before coupons/points, with free shipping strictly above its saved threshold. Google Routes, dispatch coordinates, slabs and distance quotes are inactive. Historical order charges are unchanged. Distance-specific material below describes a retained, inactive implementation until a future explicitly approved re-enable. See current flat shipping reference.

Customer addresses and checkout review (local implementation)

Follow-on local phase: configurable shipping settings supplies the newly specified flat-fee policy. Its saved configuration replaces the shipping placeholder when valid; payment and order placement remain unavailable.

This phase connects the existing Bag to authenticated address selection and a trusted merchandise review. All changes and rule preparations stay local: no commit, push, Vercel deployment, Firebase publication or production customer mutation is part of this phase.

Routes and design

- My Account has an **Addresses** entry at `/account/addresses`.
- Bag has **Review checkout**, opening `/checkout`.
- Both routes use the existing Firebase customer session and allowlisted login/registration return flow. Empty checkout shows only an empty Bag state and Continue shopping.
- The checkout's maroon progress strip, ivory background, address/item cards and desktop summary columns reuse the existing screen structure. Simulated payments, order creation, taxes, coupons and delivery promises have been removed from that screen. Native modal dialogs manage focus, Escape, background interaction and focus restoration.

Address forms require recipient name, mobile, PIN, building/street, locality, city and State/Union Territory; landmark and label are optional. Country is India. State/UT options reference the National Portal government directory. No city/PIN pairing, serviceability or delivery date is inferred. Mobile normalization reuses the existing libphonenumber policy, defaulting to India; explicit valid international country codes are accepted. Recipient edits never change the private customer profile. New forms prefill profile name/mobile without replacing fields the user has touched.

Private schema

`customerShopping/{uid}/addresses/{32-hex address ID}`:

Fields	Meaning
`recipientName`, `mobileNumber`, `pinCode`	Required contact; mobile stored as normalized E.164
`line1`, `locality`, `city`, `state`, `country`	Required address; country is `India`
`landmark`, `label`	Optional strings, stored as empty strings when omitted in the form
`revision`	Integer edit/delete conflict protection
`createdAt`, `updatedAt`	Server timestamps; edits preserve creation time

`customerShopping/{uid}/preferences/delivery` holds `defaultAddressId` (ID or null), `revision` and server `updatedAt`. There are no competing per-address default flags. Every mutation reads/writes this single pointer transactionally. The first address becomes default; changing default is atomic; deleting it chooses the lexicographically smallest remaining address ID; deleting the last leaves null. At most **10 addresses per account** are permitted, including concurrent creation requests.

Existing server-only `customerShopping/{uid}/operations/{operationId}` receipts make uncertain save/default/delete retries idempotent. Receipts contain operation hashes and outcomes/IDs, never address text or phone numbers. All UID paths derive from verified Firebase tokens with revocation and project checks. Administrators retain the separate `admin: true` plus enabled `adminAccess/{uid}` requirement for their operations. Administrators do not gain a customer address listing route.

The existing catalog function handles `addresses:list`, `addresses:save`, `addresses:default`, `addresses:delete` and `checkout:review`. It rejects unknown fields, supplied UIDs, prices, totals, stock and product payloads. Writes validate types, lengths, country, phone, PIN, allowed state values, revisions and IDs. No extra serverless function or configuration variable is introduced.

Read-only checkout operation

`checkout:review` accepts only `{action:'checkout:review', addressId: ownedAddressId | null}`. A Firestore read transaction resolves the current owned Bag, selected owned address, published products, enabled categories, variants, stock and current prices. It returns item-specific validation issues separately from HTTP/network failures. Missing/invalid quantities, variants, unavailable products and insufficient stock are retained as issues; nothing is removed, substituted or reduced.

The JSON result includes a stable content fingerprint, owned address projection, public product projections, item issues and an exact integer-paise merchandise subtotal encoded as a decimal string. If some lines cannot be priced, it explicitly marks the subtotal incomplete and the UI labels it **priced items**. Insufficient-stock items still show their requested quantity and current merchandise price, with a blocking issue. The review is not a payable quote or an authorization token.

The client compares newly returned server prices against prices previously displayed in the Bag/review and asks the customer to review changes. A hard refresh has no trustworthy previous displayed price, so it presents the current price without inventing a change. It never supplies price expectations to the server. Responses are invalidated on account, Bag, selected-address or address-revision changes; superseded requests are aborted and ignored. Editing/adding within checkout updates selection; the mutation returns a book revision so a delayed list cannot reset a newly saved selection; remote edits refresh the review and remote deletion selects the default/remaining address with an explicit notice.

Review performs no writes: no reservation, inventory decrement, order creation or Bag clearing. Future payment/order creation must independently revalidate address ownership, prices, stock, totals and approved shipping.

Shipping and payments

No approved shipping coverage, charges or threshold were found. Existing zero-valued placeholder store settings are not an approved shipping policy and are not used. The UI shows **Shipping: To be confirmed**, never zero/free shipping. Merchandise subtotal is not labelled final payable total. Payment and Place order remain unavailable, with concise preparation copy.

Still needed from the owner: delivery coverage/exclusions, shipping charges/thresholds, any applicable tax treatment, payment provider/configuration and approved order/payment policies. This phase adds no fictional values for these inputs.

Local testing and future release

Use `npm.cmd run lint`, `npm.cmd test`, `npm.cmd run build`, `npm.cmd run test:admin` and optional `npm.cmd run test:catalog-runtime`. The emulator suite includes `delivery-emulator.test.ts` and `delivery-browser.test.py`, guarded to loopback Auth/Firestore only. Browser fixtures test six widths, failure recovery, modal focus, checkout selection, remote changes, stale responses and account isolation. Existing customer/admin/category/product/appearance/shopping regressions remain included.

Manual local acceptance: sign in using a loopback test account; open My Account -> Addresses; add/edit/default/delete addresses; add a real emulator product to Bag; open Review checkout; select/add/edit an address; verify current subtotal, item issues, pending shipping and disabled payment. Refresh or use a second emulator browser context to check private persistence and remote changes.

Narrow **unpublished** Firestore changes allow owner-only address collection reads and a get/listen to the exact delivery preference document. All browser writes remain denied. No new indexes are required. A future authorized release must review the then-deployed rules, merge only these additions, run tests and publish them together with the application release. This implementation does not publish or verify production address access.

Changed files

- `src/App.tsx`, `src/views/BagView.tsx`, `src/views/CheckoutView.tsx`, `src/customer/CustomerAccount.tsx`, `src/customer/policy.ts`: routing, account entry, Bag action and the real review screen.
- `src/delivery/`: address model/validation and unit tests, API client, private subscriptions, address manager/editor, customer gate and stale-safe checkout review hook.
- `server/delivery.ts`, `server/catalog.ts`, `server/handler.ts`: trusted address operations/review and existing function dispatch/diagnostics.
- `firestore.rules`: local owner-only address/default-pointer reads.
- `package.json`, `scripts/admin-emulator-suite.ts`, `scripts/customer-browser.test.py`, `scripts/storefront-browser.test.py`, `scripts/delivery-emulator.test.ts`, `scripts/delivery-browser.test.py`, `scripts/delivery-fixture-control.ts`: tests, fixtures and updated regression expectations.
- `docs/DELIVERY_REVIEW.md`: schema, behavior, local acceptance and future release/business dependencies.